

HexStrike AI

AI-Powered Cybersecurity Learning Platform



Made by:

Aurelia Sukianto (SL25225007)

El Abdi Jaber (SL25225001)

Noah Kalyesubula (SL25225010)

Steven (SL25225003)

Department: Software Engineering

Teacher:

Guo Yan (郭燕)

As Part of 2025 Engineering Practice (2025 工程实践) Course's Final Report

Table of Contents

Abstract	5
1. Introduction	6
1.1 Project Overview	6
1.2 Problem Statement	6
1.3 Objectives	6
1.4 Scope	6
1.5 Target Users	6
2. System Requirements	7
2.1 Functional Requirements	7
2.2 Non-Functional Requirements	7
2.3 Hardware Requirements	8
2.4 Software Requirements	8
3. System Architecture	9
3.1 High-Level Architecture	9
3.2 Component Architecture	9
3.3 Data Flow: Tool Execution	10
3.4 Request Routing and Proxy	11
4. Technology Stack	11
4.1 Frontend Technologies	11
4.2 Backend Technologies	12
4.3 DevOps and Infrastructure	13
4.4 External Integrations	13
5. Database Design	14
5.1 Entity-Relationship Diagram	14
5.2 Table Descriptions	14
5.2.1 users	14
5.2.2 user_progress	15
5.2.3 quiz_attempts	15
5.2.4 pentest_sessions	16
5.2.5 flashcard_sets	17
5.2.6 user_flashcard_progress	17
5.3 Design Decisions	18

6. API Design	18
6.1 Authentication Endpoints.....	18
6.2 Tool Execution Endpoints.....	18
6.3 Learning Endpoints.....	18
6.4 User Progress Endpoints	19
6.5 Learning Endpoints.....	19
7. Frontend Design and User Interface	19
7.1 Design Philosophy	19
7.2 Routing Structure	19
7.3 Table Descriptions.....	20
7.3.1 Home Page	20
7.3.2 Login Page	20
7.3.3 Register Page	20
7.3.4 Dashboard Page	20
7.3.5 Lab Page (Pentest Lab)	20
7.3.6 Learn Page (Learning Center).....	20
7.3.7 Profile Page.....	20
7.3.8 Pentest Session Detail Page	21
7.4 Component Hierarchy	21
7.5 State Management.....	21
7.6 API Service Layer	22
8. Key Features and Functionalities.....	22
8.1 User Authentication and Authorisation.....	22
8.2 Penetration Testing Tool Execution	22
8.3 Finding Parser and Severity Classification	22
8.4 Interactive Learning Modules	22
8.5 Quiz System.....	23
8.6 Flashcard System with Spaced Repetition.....	23
8.7 Gamification System.....	23
8.8 AI Hint Assistant.....	24
8.9 Web Terminal (Xterm.js)	24
8.10 AI Service (Prepared, Not Yet Connected)	24
9. Security Considerations	25

9.1 Authentication Security.....	25
9.2 CORS Configuration.....	25
9.3 Input Validation.....	25
9.4 Route Protection.....	26
9.5 Environment Variable Management.....	26
9.6 Known Security Concerns for Production	26
9.7 Ethical Considerations	26
10. Deployment and DevOps.....	26
10.1 Current Deployment Model	26
10.2 Service Orchestration (run-all.sh)	27
10.3 Docker (Placeholder)	27
10.4 Python Environment Setup	27
10.5 CI/CD.....	27
11. Testing and Validation.....	27
11.1 Current Testing Status	27
11.2 Manual Testing Capabilities.....	27
11.3 Known Limitations	28
12. Future Enhancements.....	28
12.1 Immediate Improvements	28
12.2 Scalability Improvements	28
12.3 Feature Enhancements	28
13. Conclusion	29
14. Appendices.....	29
14.1 Appendix A: Complete File Structure.....	29
14.2 Appendix B: API Endpoint Summary.....	29
14.3 Appendix C: Glossary	29
14.4 Appendix D: Screenshots.....	30

Abstract

HexStrike AI is a cybersecurity education platform powered by artificial intelligence that teaches penetration testing and ethical hacking through an interactive and gamified learning experience, combining real security tool execution with AI generated explanations from OpenAI GPT-4o, structured learning modules, quizzes, flashcards, and a reward system built around experience points, achievements, and activity streaks. The platform uses a three tier client server design, with a React single page frontend, a FastAPI backend that handles business logic and data storage, and a service layer made up of the HexStrike Server running more than 150 security tools, the OpenAI API, and a SQLite database, all connected through a REST API with five router groups and JWT authentication. It already includes forty two security tools that run in the browser with live progress updates, ten learning modules with quizzes and flashcards, a working reward system, a hint assistant, and a responsive dark themed interface built for cybersecurity learning. The platform works well for local educational use, but it is not yet ready for production, since containerization, automated testing, CI/CD pipelines, and a production ready database are still planned, though the current architecture is solid enough to support these additions without major redesign.

摘要

HexStrike AI 是一个由人工智能驱动的网络安全教育平台，通过互动和游戏化的学习方式教授渗透测试与道德黑客技术，将真实安全工具的执行，由 OpenAI GPT-4o 提供支持的人工智能讲解，结构化学习模块，测验，闪卡，以及基于经验值，成就和连续学习天数的奖励机制结合在一起，整体采用三层客户端与服务器架构，前端是 React 单页应用，后端使用 FastAPI 构建并负责业务逻辑和数据存储，服务层包括可运行超过一百五十种安全工具的 HexStrike Server，OpenAI API，以及 SQLite 数据库，各部分通过由五个路由组构成并使用 JWT 身份验证的 REST API 相连。该平台目前已经实现四十二个可在浏览器中运行并提供实时进度反馈的安全工具，十个配有测验和闪卡的学习模块，一个可正常运行的奖励机制，一个提示助手，以及一个适合网络安全学习场景的响应式深色主题界面，目前适合本地教学使用，但距离生产环境部署还有一定距离，容器化，自动化测试，CI/CD 流程，以及适合生产环境的数据库仍在规划之中，不过目前的架构基础良好，可以在不进行重大重构的情况下支持这些扩展。

1. Introduction

1.1 Project Overview

HexStrike AI is a full-stack, AI-powered cybersecurity education platform designed to teach penetration testing and ethical hacking in an interactive, gamified environment. The platform combines real-world security tool execution with artificial intelligence to provide contextual explanations, structured learning modules, and hands-on lab exercises. This combination makes it possible for learners to engage with genuine security tools while receiving AI-assisted guidance, bridging a gap that conventional learning environments rarely address.

1.2 Problem Statement

Traditional cybersecurity education frequently struggles to engage learners meaningfully. Beginners approaching penetration testing often face a steep learning curve, with few guided environments that allow practice using real security tools. There is also a well-documented gap between theoretical knowledge and the practical skills employers actually need. When students do use security tools, they often receive no intelligent feedback to help them understand what the results mean or what they did wrong.

HexStrike AI addresses these gaps by integrating more than 150 security tools, AI-powered explanations using OpenAI GPT-4o, interactive learning modules with embedded quizzes, and a gamification system that tracks experience points (XP), achievements, and activity streaks. Together, these features create a learning environment that is both educationally structured and practically engaging.

1.3 Objectives

The platform pursues six core objectives. First, it provides a web-based environment for learning ethical hacking and penetration testing. Second, it integrates real penetration testing tools that are executable directly from the browser. Third, it uses AI via GPT-4o to translate scan results into beginner-friendly language. Fourth, it offers structured learning modules supported by quizzes and flashcards. Fifth, it motivates learners through a gamification framework built around XP, levels, achievements, and streaks. Sixth, it supports hands-on practice sessions targeting a deliberately vulnerable machine, Metasploitable 2.

1.4 Scope

The platform is designed for educational use in controlled laboratory environments, specifically Kali Linux virtual machines targeting Metasploitable 2. It is not intended as a production-grade offensive security product, and its design decisions reflect this educational context throughout.

1.5 Target Users

HexStrike AI is designed for four primary user groups: students learning ethical hacking and cybersecurity as part of formal or self-directed study; penetration testing trainees seeking structured practice with real tools; cybersecurity enthusiasts who are self-learning outside academic institutions; and instructors who run cybersecurity lab courses and need a platform that structures both tool execution and conceptual learning.

2. System Requirements

2.1 Functional Requirements

The following table outlines the thirteen functional requirements identified for the platform, together with their assigned priority levels following a MoSCoW classification framework.

Table 1. Functional Requirements and Priority Levels.

ID	Requirement	Priority
FR-01	User registration and authentication with JWT	Must
FR-02	Execute 42+ security tools against target systems	Must
FR-03	Display real-time scan progress in the browser	Must
FR-04	Parse and display scan findings with severity levels	Must
FR-05	AI-powered explanations of scan results (GPT-4o)	Should
FR-06	10 interactive learning modules with quizzes	Must
FR-07	Flashcard system with spaced repetition mastery tracking	Should
FR-08	Gamification: XP, achievements, activity streaks	Should
FR-09	Dashboard with analytics and progress visualisation	Must
FR-10	Pentest session management (create, view, delete, export)	Must
FR-11	Contextual hint assistant for lab exercises	Should
FR-12	Web-based interactive terminal (Xterm.js)	Should
FR-13	Study notes per module (persisted in localStorage)	Could

2.2 Non-Functional Requirements

Non-functional requirements describe the quality and operational characteristics the system must exhibit. These cover performance, usability, security, availability, and compatibility, as shown below.

Table 2. Non-Functional Requirements by Category.

ID	Requirement	Details
NFR-01	Performance	Tool execution timeout: 600s per tool; dashboard auto-refresh: 30s

NFR-02	Usability	Dark-themed UI, responsive design, mobile hamburger menu
NFR-03	Security	bcrypt/PBKDF2 password hashing, JWT auth, CORS configuration
NFR-04	Availability	Local development server (not production-hardened)
NFR-05	Compatibility	Designed for Kali Linux 2024.1+; browser-based frontend

2.3 Hardware Requirements

The hardware specifications below represent the minimum and recommended configurations for running the platform effectively on a local Kali Linux machine.

Table 3. Minimum and Recommended Hardware Specifications.

Component	Minimum	Recommended
CPU Cores	2	4
RAM	4 GB	8 GB
Storage	20 GB	30 GB
OS	Kali Linux 2024.1+	Kali Linux Rolling

2.4 Software Requirements

The software dependencies required to run HexStrike AI are listed below. Several of these are standard tools already present on Kali Linux installations, while others require explicit installation steps.

Table 4. Required Software Dependencies and Versions.

Software	Version
Python	3.11+ (install script targets 3.13)
Node.js	20.x
npm	9.x+
SQLite	Built-in with Python
HexStrike Server	Sibling repository (../hexstrike-server)
Metasploitable 2 VM	For target practice

3. System Architecture

3.1 High-Level Architecture

HexStrike AI follows a three-tier client-server architecture. The three tiers consist of a React single-page application on the frontend, a FastAPI backend that handles business logic and data persistence, and an external service layer comprising the HexStrike Server for tool execution, an OpenAI API connection for AI functionality, and a SQLite database for data storage. Communication between the frontend and backend occurs through a REST API on port 8000 and WebSocket connections for real-time terminal I/O. The diagram below illustrates how the tiers interact.

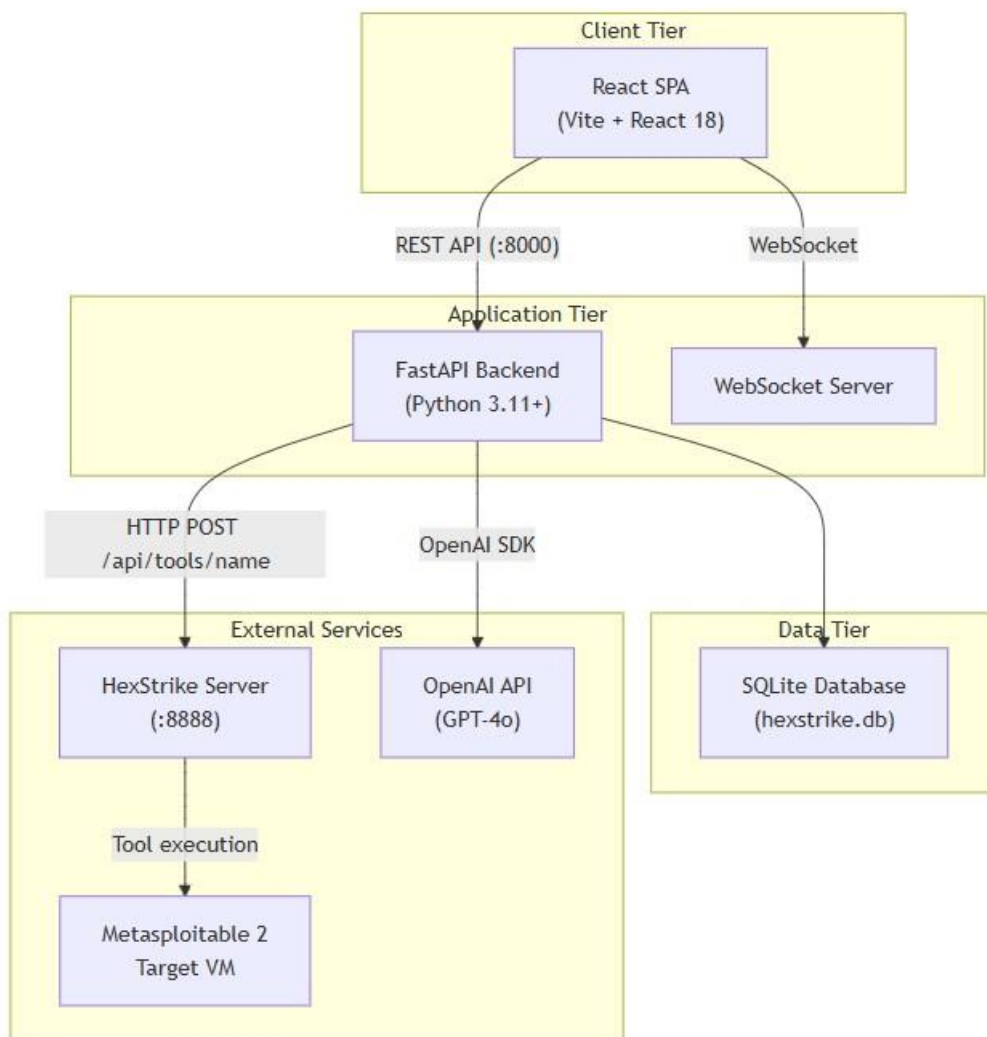


Figure 1. High-Level System Architecture

3.2 Component Architecture

The backend is organised around a central `main.py` entry point that registers five routers (auth, tools, learning, analytics, and hints) and their associated services. Each router communicates with a dedicated service layer component: the tools router uses `ToolsService` and `ResultParser`; the learning router uses `LearningService`; and both the hints router and the

websocket handler share the LLMService and WebSocketManager. The frontend is structured around App.jsx, which mounts a global Navbar, a floating HintAssistant, and an ErrorBoundary wrapper, with a ProtectedRoute component guarding the authenticated pages. The diagram below shows the full component structure.

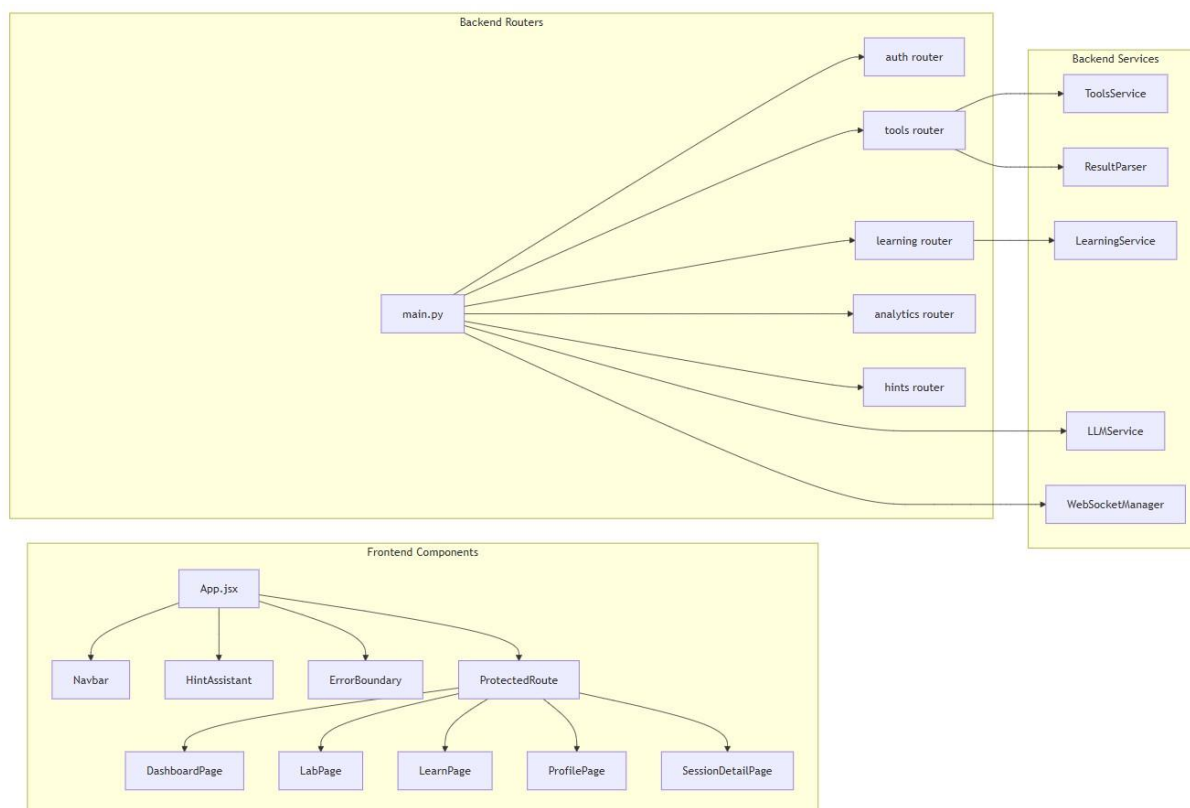


Figure 2. Backend Routers and Frontend Component Hierarchy

3.3 Data Flow: Tool Execution

The sequence diagram below illustrates the full flow of a typical pentest batch session. The user selects tools and enters a target IP in the browser. The React frontend sends a POST request to `/api/tools/batch` on the FastAPI backend, which creates a `PentestSession` record in the database and returns a session ID. The backend then loops through each selected tool, sending POST requests to the HexStrike Server, which executes the security tool against the Metasploitable 2 target and returns raw output. The backend parses findings by severity and updates the session record. Meanwhile, the frontend polls `GET /api/tools/session/{id}` every two seconds, updating the progress bar and console display until all tools are complete.

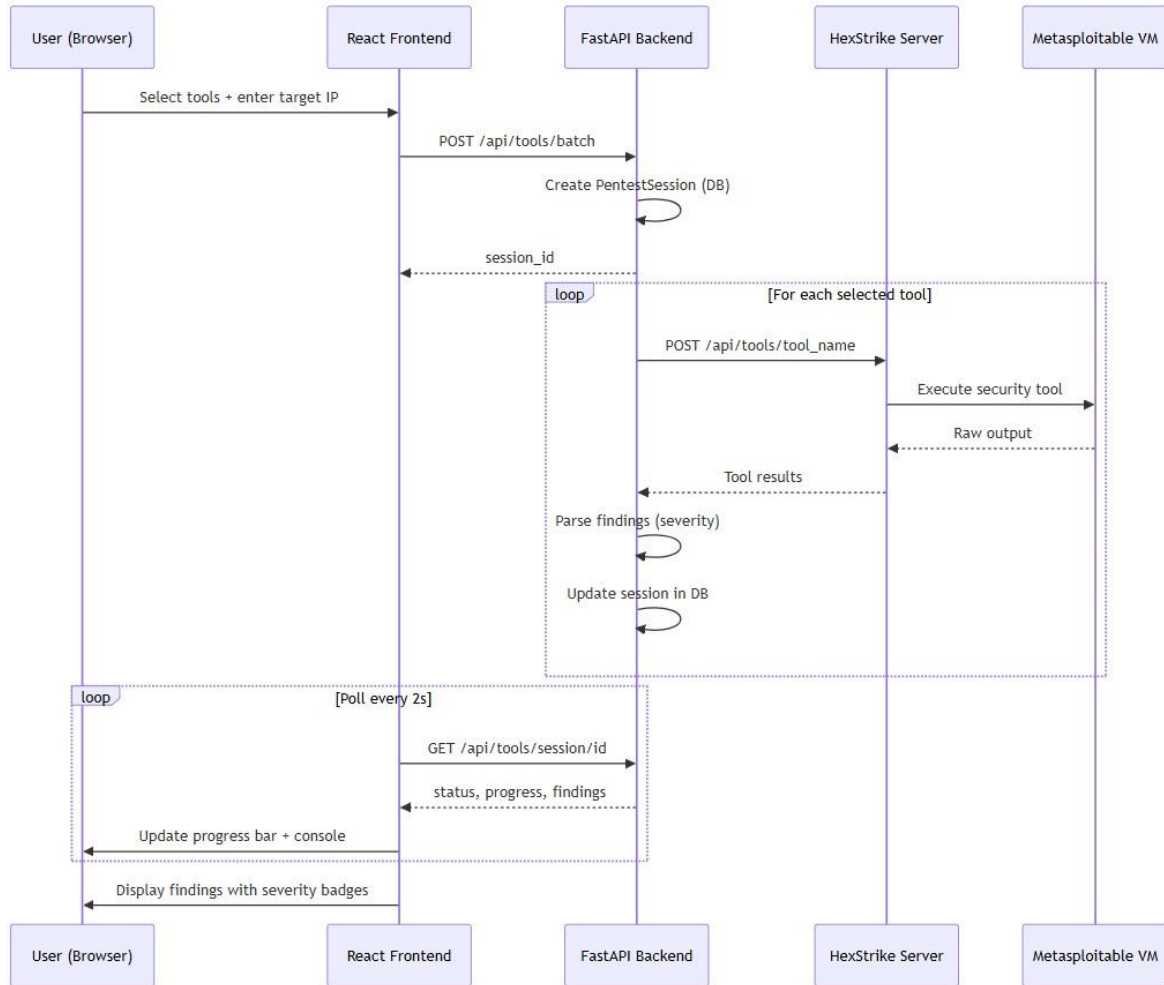


Figure 3. Tool Execution Sequence Diagram

3.4 Request Routing and Proxy

The Vite development server proxies all `/api/*` requests to the FastAPI backend at `http://localhost:8000`. The special `/api/health` path is rewritten to `/health` on the backend. This arrangement allows the frontend to make API calls without encountering CORS issues during development, since all requests appear to originate from the same host from the browser's perspective.

4. Technology Stack

4.1 Frontend Technologies

The frontend is built entirely in React and relies on a set of well-established libraries to manage routing, styling, data visualisation, and terminal emulation. The table below lists all frontend dependencies along with their versions and roles.

Table 5. Frontend Technology Stack and Versions.

Technology	Version	Purpose
React	18.2.0	Component-based UI framework
Vite	5.0.0	Build tool and dev server with HMR
React Router DOM	6.30.3	Client-side SPA routing
Tailwind CSS	3.3.5	Utility-first CSS framework
Xterm.js	5.3.0	Web-based terminal emulator
xterm-addon-fit	0.8.0	Auto-resize terminal to container
xterm-addon-web-links	0.9.0	Clickable links in terminal
Axios	1.6.0	HTTP client for API calls
Recharts	2.15.4	Data visualisation charts
Lucide React	0.294.0	Icon library
React Hot Toast	2.6.0	Toast notification system
React Markdown	10.1.0	Markdown rendering
React Circular Progressbar	2.2.0	Circular progress indicators

4.2 Backend Technologies

The backend is written in Python using FastAPI as the web framework. It integrates authentication, database access, AI capabilities, and tool communication through the libraries listed below.

Table 6. Backend Technology Stack and Versions.

Technology	Version	Purpose
FastAPI	0.115.6	Async Python web framework
Uvicorn	0.34.0	ASGI server
SQLAlchemy	2.0.36	ORM for database access
Pydantic	2.10.4	Data validation and serialisation
Pydantic Settings	2.6.1	Environment-based configuration
python-jose	3.3.0	JWT token encoding/decoding (HS256)
Passlib	1.7.4	Password hashing (PBKDF2-SHA256)

OpenAI	1.59.3	GPT-4o API client
httpx	0.28.1	Async HTTP client (HexStrike comms)
websockets	14.1	WebSocket protocol support
email-validator	2.2.0	Email validation for registration
SQLite	Built-in	Embedded relational database

4.3 DevOps and Infrastructure

The current DevOps setup is lightweight and intended for local development use. Docker containerisation is acknowledged in the project structure but has not been implemented. The primary orchestration mechanism is a shell script that starts all three services together.

Table 7. DevOps Tooling and Implementation Status.

Technology	Purpose
Docker (placeholder)	Containerisation files exist but are empty
Docker Compose (placeholder)	Multi-service orchestration (not yet implemented)
Shell scripts	run-all.sh for local service orchestration
Git	Version control

4.4 External Integrations

Three external services are integrated into the platform. The HexStrike Server executes penetration testing tools and communicates with the backend over HTTP. OpenAI GPT-4o provides AI capabilities through the Python SDK. Metasploitable 2 serves as the practice target and is accessed directly over the network.

Table 8. External Service Integration Summary.

Service	Purpose	Integration Method
HexStrike Server (:8888)	Executes 150+ penetration testing tools	HTTP POST to /api/tools/{name}
OpenAI GPT-4o	AI explanations, quiz generation, progress analysis	OpenAI Python SDK (async)
Metasploitable 2 VM	Vulnerable target for practice	Network target (typically 192.168.56.101)

5. Database Design

5.1 Entity-Relationship Diagram

The database schema is centred on the users table, which links to five dependent tables through foreign-key relationships with cascade delete. The entity-relationship diagram below shows all tables and their associations.

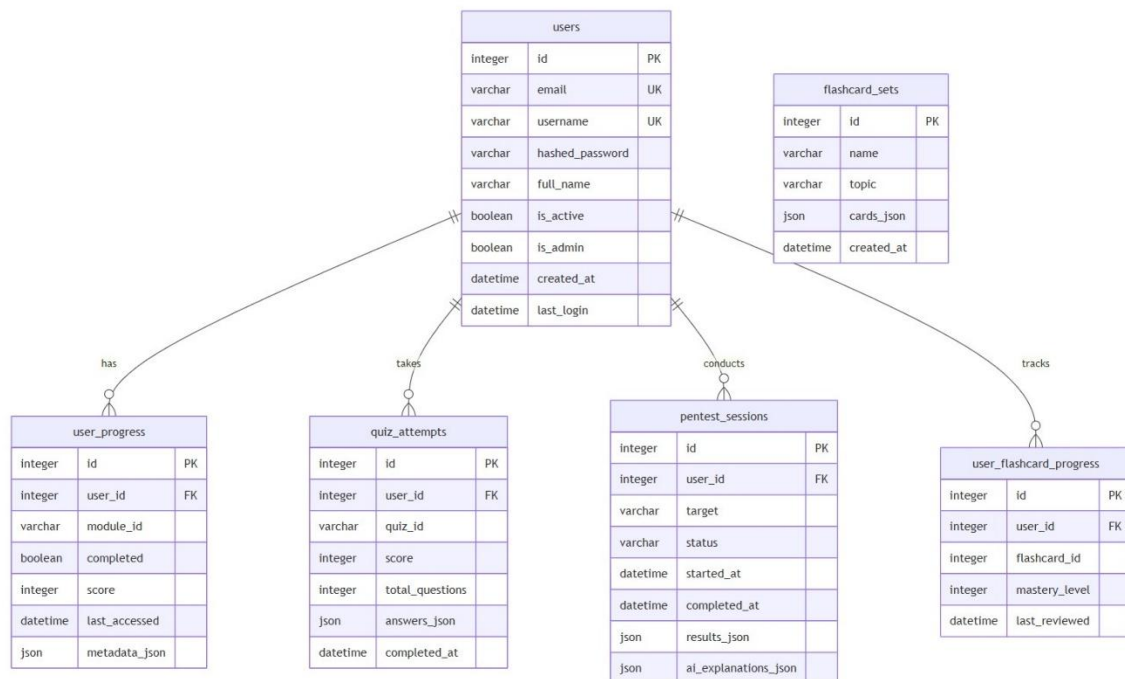


Figure 4. Entity-Relationship Diagram

5.2 Table Descriptions

5.2.1 users

The users table stores credentials and profile data for every registered user. All dependent tables reference it through a foreign key with cascade delete, so removing a user removes all their associated records.

Table 9. Column Definitions for the users Table.

Column	Type	Constraints	Description
id	Integer	PK, auto-increment	Unique user identifier
email	String	Unique, NOT NULL, indexed	User email address

username	String	Unique, NOT NULL, indexed	Display username
hashed_password	String	NOT NULL	PBKDF2-SHA256 hashed password
full_name	String	Nullable	Optional full name
is_active	Boolean	Default: True	Account active status
is_admin	Boolean	Default: False	Admin role flag
created_at	DateTime	Default: utcnow	Registration timestamp
last_login	DateTime	Nullable	Last login timestamp

5.2.2 user_progress

This table records each user's interaction with a learning module, including completion status, best quiz score, and the last time the module was accessed.

Table 10. Column Definitions for the user_progress Table.

Column	Type	Constraints	Description
id	Integer	PK	Unique progress record
user_id	Integer	FK → users.id	Owning user
module_id	String	Indexed	Learning module identifier
completed	Boolean	Default: False	Completion status
score	Integer	Default: 0	Best quiz score (percentage)
last_accessed	DateTime	Default: utcnow	Last access timestamp
metadata_json	JSON	Default: {}	Additional metadata

5.2.3 quiz_attempts

Every quiz submission is stored here with its score, total question count, and a JSON blob of submitted answers together with their correctness flags.

Table 11. Column Definitions for the quiz_attempts Table.

Column	Type	Constraints	Description
id	Integer	PK	Unique attempt record
user_id	Integer	FK → users.id	Owning user
quiz_id	String	Indexed	Quiz/module identifier
score	Integer	-	Number of correct answers
total_questions	Integer	-	Total questions in the quiz
answers_json	JSON	-	Submitted answers with results
completed_at	DateTime	Default: utcnow	Submission timestamp

5.2.4 pentest_sessions

This table records each pentest batch session, including the target, status lifecycle (pending, running, completed, failed), timing, and a JSON blob of all tool call results and parsed findings.

Table 12. Column Definitions for the pentest_sessions Table.

Column	Type	Constraints	Description
id	Integer	PK	Unique session record
user_id	Integer	FK → users.id	Owning user
target	String	-	Target IP/domain
status	String	Default: pending	Session status: pending/running/completed/failed
started_at	DateTime	Default: utcnow	Session start time
completed_at	DateTime	Nullable	Session end time
results_json	JSON	Default: {}	Full results including tool calls and findings
ai_explanations_json	JSON	Default: {}	AI-generated explanations (defined, not yet populated)

5.2.5 flashcard_sets

This table defines flashcard sets by topic. Note that although the table is defined in the schema, flashcard data is currently served from in-memory data within LearningService rather than from the database.

Table 13. Column Definitions for the flashcard_sets Table.

Column	Type	Constraints	Description
id	Integer	PK	Unique flashcard set
name	String	NOT NULL	Set name
topic	String	Indexed	Topic category
cards_json	JSON	-	Array of front/back card pairs
created_at	DateTime	Default: utcnow	Creation timestamp

5.2.6 user_flashcard_progress

This table records each user's mastery rating for individual flashcards, supporting the spaced repetition algorithm.

Table 14. Column Definitions for the user_flashcard_progress Table.

Column	Type	Constraints	Description
id	Integer	PK	Unique progress record
user_id	Integer	FK → users.id	Owning user
flashcard_id	Integer	-	Individual card ID
mastery_level	Integer	Default: 0	Mastery rating (1=hard, 3=medium, 5=easy)
last_reviewed	DateTime	Nullable	Last review timestamp

5.3 Design Decisions

Three key design choices shaped the database structure. First, SQLite was selected because it requires no separate server process and is well suited to single-user or small-group educational environments where simplicity matters more than scalability. Second, JSON columns are used for flexible, schema-less storage of tool results, quiz answers, and metadata, avoiding the need for additional normalised tables. Third, cascade delete is applied throughout, so that removing a user automatically removes all their progress records, quiz attempts, and pentest sessions.

6. API Design

The backend exposes a RESTful API under the `/api/` path, organised into five router groups. All authenticated endpoints require a Bearer JWT token in the Authorization header. FastAPI auto-generates interactive API documentation at `/docs` (Swagger UI) and `/redoc` (ReDoc).

6.1 Authentication Endpoints

The authentication router handles user registration, login, and profile retrieval through the three endpoints listed below.

Table 15. Authentication Router Endpoint Summary.

Method	Endpoint	Auth	Description
POST	<code>/api/auth/register</code>	None	Register a new user account
POST	<code>/api/auth/token</code>	None	Login and receive a JWT access token
GET	<code>/api/auth/me</code>	JWT	Retrieve the current authenticated user profile

6.2 Tool Execution Endpoints

The tools router manages all interactions with the HexStrike Server. It exposes endpoints for executing individual tools, launching batch sessions across multiple tools, checking session progress, retrieving session history, and deleting sessions. Tool execution is handled asynchronously using FastAPI's BackgroundTasks, and the frontend polls for progress every two seconds.

6.3 Learning Endpoints

The learning router provides access to module content, quiz submission, flashcard retrieval, and quiz history. Module content and quiz questions are stored in-memory in the LearningService rather than in the database, which simplifies deployment but limits dynamic content updates. Quiz results are persisted in the database upon submission.

6.4 User Progress Endpoints

Progress endpoints allow the frontend to retrieve and update a user's learning progress, including XP totals, completed modules, activity streaks, and achievement status. The backend recalculates aggregate values such as total XP on each request.

6.5 Learning Endpoints

A health endpoint at /health checks the status of the backend, HexStrike Server, and database simultaneously. A WebSocket endpoint at /ws/{client_id} provides a real-time terminal channel for interactive tool output streaming. The root endpoint at / returns a simple platform status confirmation.

7. Frontend Design and User Interface

7.1 Design Philosophy

The frontend adopts a dark-themed cybersecurity aesthetic throughout. The primary background uses a deep navy colour (#1a1a2e), with card backgrounds in dark blue-grey (#16213e) and input fields in midnight blue (#0f3460). The accent colour is an indigo-violet (#6c63ff), with gradient accents at 135 degrees between #667eea and #764ba2. Severity colours follow a traffic-light system: critical and high findings use red (#ef4444), medium findings use amber (#f59e0b), low and successful states use green (#10b981), and informational items use blue (#3b82f6).

7.2 Routing Structure

The application uses React Router DOM for client-side routing. Three routes are publicly accessible without authentication. All other routes are protected and redirect unauthenticated users to the login page.

Table 16. Application Routes and Access Levels.

Route	Page Component	Access
/	HomePage	Public
/login	LoginPage	Public
/register	RegisterPage	Public
/dashboard	DashboardPage	Protected
/dashboard/sessions/:id	PentestSessionDetailPage	Protected
/lab	LabPage	Protected
/learn	LearnPage	Protected
/profile	ProfilePage	Protected

7.3 Table Descriptions

7.3.1 Home Page

The home page presents a gradient background with the platform title and tagline. It includes a backend health check status indicator and two call-to-action buttons. Authenticated users are automatically redirected to the dashboard upon visiting this page.

7.3.2 Login Page

The login page provides a simple form with username and password fields. On successful authentication, the JWT token is stored in localStorage, the user profile is fetched, and the user is redirected to the dashboard.

7.3.3 Register Page

The registration page collects a username, email address, an optional full name, and a password with confirmation. Client-side validation checks that passwords match and meet the minimum six-character requirement. Successful registration redirects the user to the login page.

7.3.4 Dashboard Page

The dashboard presents five statistics cards showing total XP, completed modules, pentest count, total findings, and average quiz score. Below the statistics, a table lists recent pentest sessions with their target, date, findings count, status, and view or delete actions. An achievements section displays five achievement cards with earned status and progress indicators. The dashboard auto-refreshes every thirty seconds.

7.3.5 Lab Page (Pentest Lab)

The lab page is the primary tool execution interface. Users enter a target IP address or domain, then select tools from a filterable and searchable list organised by category. Once a batch is launched, a real-time progress bar updates every two seconds, showing per-tool status in a console output panel. A right sidebar displays recent session history.

7.3.6 Learn Page (Learning Center)

The learn page displays the ten structured learning modules in a left sidebar with circular progress indicators, difficulty badges, and XP values. Users can search and filter modules by difficulty level. Selecting a module opens six content tabs: Overview (long description and learning objectives), Quiz (multiple-choice questions with 70% pass threshold), Lab Exercise (step-by-step instructions linked to the Lab page), History (quiz attempt history with scores), Flashcards (interactive flip cards with mastery tracking), and Notes (free-text study notes persisted in localStorage).

7.3.7 Profile Page

The profile page shows the user's statistics, recent activity, earned achievements, and study notes. The user avatar displays the first letter of the username. The profile includes username, email, full name, and member-since date.

7.3.8 Pentest Session Detail Page

The session detail page presents four metric cards showing status, tool count, findings count, and start date. A tool calls section lists per-tool timing, HTTP status, findings count, and expandable raw output. A findings section presents each finding with its title, originating tool, severity badge, and description. Users can download the full session as JSON or delete it from this page.

7.4 Component Hierarchy

The application's component tree is rooted at App.jsx, which mounts ErrorBoundary, AuthProvider (providing AuthContext), and AppContent. AppContent renders the global Navbar, the floating HintAssistant, and the route tree. Public routes render HomePage, LoginPage, and RegisterPage directly. Protected routes are wrapped in ProtectedRoute, which guards DashboardPage, PentestSessionDetailPage, LabPage, LearnPage (with its Flashcards sub-component), and ProfilePage. Components are organised into six functional categories, as shown in the table below.

Table 17. Frontend Component Categories.

Category	Components	Purpose
Common	Navbar, ErrorBoundary, LoadingSpinner	Shared layout and utilities
Auth	Login, Register, ProtectedRoute	Authentication UI and route guarding
Dashboard	Dashboard, ProgressChart, StatsCard	Analytics visualisation
PentestLab	PentestConsole, ResultsViewer, SessionHistory, ToolExecutor	Tool execution UI
Learning	Flashcards, LearningPath, QuizModule	Educational content delivery
Terminal	XTerminal	Xterm.js-based interactive terminal

7.5 State Management

Authentication state is managed globally through AuthContext (a React Context), which tracks user, token, isAuthenticated, and loading state and exposes login() and logout() methods. Each page manages its own data fetching using useState and useEffect hooks. The platform defines four custom hooks: useAuth() for accessing the auth context; usePentestSession() for session management logic; usePersistedState() for localStorage-backed state persistence; and useWebSocket() for WebSocket connection management.

7.6 API Service Layer

Frontend API calls are centralised in four service modules. The base `api.js` handles generic HTTP methods with automatic auth header injection and 401 redirect logic. `authService.js` provides `login()`, `register()`, `getMe()`, and `logout()`. `learningService.js` covers `getModules()`, `getModuleContent()`, `submitQuiz()`, `getFlashcards()`, `reviewFlashcard()`, and `getQuizHistory()`. `toolService.js` manages `executeTool()`, `executeBatch()`, `getSessionStatus()`, `getHistory()`, `deleteSession()`, and `clearHistory()`.

8. Key Features and Functionalities

8.1 User Authentication and Authorisation

User registration collects an email, username, and password, with the password hashed using PBKDF2-SHA256 before storage. Duplicate email and username checks are performed at registration time. Login follows the OAuth2 password flow and issues a JWT access token using HS256 signing with a seven-day expiry period. On the frontend, a `ProtectedRoute` component intercepts unauthenticated navigation and redirects to the login page. On the backend, a `get_current_user` dependency validates the JWT token on every request to a protected endpoint. Tokens are stored in `localStorage`, and the application performs an automatic logout when it receives a 401 response.

8.2 Penetration Testing Tool Execution

The platform registers 42 penetration testing tools across nine categories: Network Discovery, Web Reconnaissance, DNS Analysis, Web Enumeration, Web Scanning, SMB Enumeration, SQL Injection, Automation, and Exploitation. Users select multiple tools and a target, then launch a batch session. The backend executes the selected tools sequentially against the HexStrike Server using asynchronous HTTP requests. The frontend polls for progress every two seconds, displaying a progress bar and per-tool status updates in a console-style output area. Intelligent target classification is built into the backend, automatically detecting whether a tool requires a URL prefix, a URL query parameter, or a raw IP address. Each tool execution has a 600-second server-side timeout and a 300-second client-side timeout.

8.3 Finding Parser and Severity Classification

Tool output is processed by the `universal_parse_findings()` function, which scans each line for indicators of security findings. It recognises keywords such as 'open', 'vulnerable', 'found', 'detected', and 'CVE-', as well as special line prefixes including '+', '-', '*', '>', and '#'. Each finding is then classified by severity through keyword matching. Lines containing 'critical' or 'rce' are classified as Critical; 'high', 'dangerous', or 'exploit' as High; 'medium', 'moderate', or 'warning' as Medium; 'low' or 'minor' as Low; and all other findings default to Info. A maximum of 50 findings per tool are retained, with a summary finding prepended to each result set.

8.4 Interactive Learning Modules

The platform includes ten learning modules covering the full penetration testing lifecycle, from basic network reconnaissance through to advanced topics such as cloud security and incident response. Each module includes a detailed description, learning objectives, a practical lab exercise with step-by-step instructions and recommended tools, quiz questions, and a flashcard deck. All module content is stored in-memory within the `LearningService`.

Table 18. Learning Modules by Difficulty and XP.

#	Module	Difficulty	XP	Duration
1	Network Reconnaissance	Beginner	100	2h
2	Vulnerability Scanning	Beginner	150	3h
3	Web Application Security	Intermediate	200	4h
4	Password Attacks and Cracking	Intermediate	175	3h
5	Wireless Network Security	Intermediate	200	4h
6	Privilege Escalation	Advanced	250	5h
7	Active Directory Security	Advanced	300	6h
8	Social Engineering	Intermediate	225	4h
9	Cloud Security	Advanced	275	5h
10	Incident Response and Forensics	Advanced	350	6h

8.5 Quiz System

Each learning module includes a quiz with multiple-choice questions, each offering four answer options. The passing threshold is set at 70%. Upon passing, the module is marked as complete, XP is awarded to the user, and the progress record is updated in the database. After submission, the user sees their answer, the correct answer, and an explanation for each question. All quiz attempts are stored with scores and timestamps. Users may retake quizzes an unlimited number of times.

8.6 Flashcard System with Spaced Repetition

The platform includes approximately 200 flashcards across all modules, with around 20 cards per module. Each card has a front face and a back face, and flipping the card is animated using CSS 3D transforms. After reviewing each card, the user rates their recall difficulty as Hard, Medium, or Easy. Cards rated Easy are removed from the current session. Hard cards are reinserted three positions later in the deck, and Medium cards reappear eight positions later, implementing a basic spaced repetition algorithm. Mastery levels are persisted in localStorage per topic and synchronised with the backend through the /api/learning/flashcards/review endpoint.

8.7 Gamification System

The gamification framework rewards learning activity through XP points, achievements, and activity streaks. Each module awards between 100 and 350 XP upon successful quiz completion. Successful pentest sessions each award 50 XP. Total XP is calculated as the sum

of all completed module XP plus the number of successful sessions multiplied by 50. Activity streaks count consecutive days on which the user made learning progress or attempted a quiz. Five achievements are available, each unlocked by meeting a specific condition, as described in the table below.

Table 19. Achievement Unlock Conditions.

Achievement	Condition
First Scan	Complete 1 pentest session
Finding Finder	Discover 10 scan findings
Quiz Master	Score 100% on 5 quizzes
Fast Operator	Complete a pentest in under 5 minutes
Course Climber	Complete 3 learning modules

8.8 AI Hint Assistant

The hint assistant is accessible from every page through a floating button in the bottom-right corner. Users select a module and optionally specify a lab exercise, then request hints progressively across three levels. Level 1 hints provide a conceptual overview, Level 2 hints offer more specific guidance, and Level 3 hints supply the full recommended command. The assistant currently operates on a static rule-based database covering four modules, with generic fallback responses for others. It presents responses in a chat-style interface with scrollable message history.

8.9 Web Terminal (Xterm.js)

The platform includes a full terminal emulator component built with Xterm.js. The terminal uses a green-on-black aesthetic with a blinking cursor in 14px Consolas font. It supports command history navigation using up and down arrow keys, and includes built-in 'clear' and 'help' commands. Command execution is handled through a callback-based onCommand prop, and the terminal auto-resizes to its container using the FitAddon.

8.10 AI Service (Prepared, Not Yet Connected)

The LLMService module defines five AI capabilities using OpenAI GPT-4o, as described in the table below. The service is fully implemented in code but has not yet been connected to any API endpoint. It is prepared for future integration and represents a clear near-term development priority.

Table 20. LLMService Methods and Parameters.

Method	Temperature	Purpose
--------	-------------	---------

explain_tool_result()	0.5	Explain scan results to beginners
optimize_tool_parameters()	0.3	Suggest optimal scan parameters
create_execution_plan()	0.4	Intelligent tool execution ordering
generate_quiz()	0.6	Dynamic quiz generation
analyze_student_progress()	0.4	Personalised learning recommendations

9. Security Considerations

9.1 Authentication Security

User passwords are hashed using PBKDF2-SHA256 with 10,000 rounds through the Passlib library before storage. JWT tokens are signed with HS256 and expire after seven days. Every authenticated endpoint validates the JWT signature and expiry on each request. The table below summarises the authentication security implementation.

Table 21. Password and Token Security Details.

Aspect	Implementation
Password hashing	PBKDF2-SHA256 with 10,000 rounds (via passlib.CryptContext)
Token format	JWT with HS256 algorithm
Token expiry	7 days (10,080 minutes)
Token storage	localStorage (client-side)
Token validation	Every authenticated endpoint validates JWT signature and expiry

9.2 CORS Configuration

CORS origins are configurable via the CORS_ORIGINS environment variable in the .env file. The defaults include <http://localhost:5173> and <http://localhost:3000>. The backend allows all methods and headers and enables credentials for authenticated requests.

9.3 Input Validation

Input validation is handled by Pydantic on the backend, which enforces data types and constraints on all incoming request payloads. Email addresses are validated using the email-

validator library at registration time. Registration also checks for duplicate email addresses and usernames. The frontend enforces a minimum password length of six characters on the client side.

9.4 Route Protection

Route protection is implemented at three layers. On the backend, the `get_current_user` dependency validates the JWT token on all authenticated endpoints and returns a 401 status on invalid or expired tokens. On the frontend, the `ProtectedRoute` component wraps all authenticated pages and redirects unauthenticated users to `/login`. At the API service layer, any 401 response automatically redirects to `/login` and clears stored tokens.

9.5 Environment Variable Management

Secrets are stored in `.env` files that are excluded from version control through `.gitignore`. The `gitignore` covers `.env`, `.env.local`, `.env.*.local`, `*.env`, and `*.db` files. Configuration is loaded at startup through `pydantic_settings.BaseSettings`, which maps environment variables to typed configuration fields.

9.6 Known Security Concerns for Production

The current implementation includes several limitations that are acceptable for a local educational environment but would require attention before any broader deployment. All communication uses unencrypted HTTP rather than HTTPS. The development frontend server binds to `0.0.0.0`, exposing it on all network interfaces. JWT tokens stored in `localStorage` are vulnerable to cross-site scripting attacks; `HttpOnly` cookies would be more secure. The default secret key in the configuration file must be changed. SQLite provides no encryption at rest. No rate limiting is applied to login or API endpoints. A GPG key file present in the repository root should also be removed.

9.7 Ethical Considerations

The platform is designed exclusively for educational use in controlled laboratory environments. Users are expected to target only authorised systems, specifically the Metasploitable 2 virtual machine included in the lab setup. The platform does not implement technical safeguards against targeting unauthorised systems, so its responsible use depends on the conduct of those who operate it.

10. Deployment and DevOps

10.1 Current Deployment Model

The platform is deployed locally on a bare-metal Kali Linux environment. Three services are orchestrated together using a shell script, each running on the port listed below.

Table 22. Service Ports and Startup Commands.

Service	Port	Start Command
FastAPI Backend	8000	<code>python run.py</code> (Uvicorn with hot-reload)

HexStrike Server	8888	python hexstrike_server.py (sibling repo)
Vite Frontend	5173	npm run dev -- --host 0.0.0.0 --port 5173

10.2 Service Orchestration (run-all.sh)

The run-all.sh script, spanning 151 lines, manages the startup and shutdown of all three services. Before launching, it validates that required files exist and checks that the required ports are free. It auto-initialises the database by creating hexstrike.db if it does not yet exist. Each service runs in its own process group using setsid. On receiving an INT, TERM, or EXIT signal, the script sends SIGTERM to all service groups, waits one second, and then sends SIGKILL to ensure clean shutdown. Output from each service is prefixed with a service name tag. The script exits immediately if any service crashes unexpectedly.

10.3 Docker (Placeholder)

The project's docker/ directory contains three empty files: Dockerfile.backend, Dockerfile.frontend, and docker-compose.yml. Docker containerisation is planned as a future improvement but has not yet been implemented. Completing these files would allow the platform to be deployed consistently across different environments.

10.4 Python Environment Setup

A separate install-py313.sh script targets Python 3.13.9 on Kali Linux. It pins all dependency versions explicitly, performs clean installs by removing old versions first, and verifies that imports succeed after installation. The script is currently hardcoded to the path /home/kali/hexstrike-ai-platform/backend and would need adjustment for other environments.

10.5 CI/CD

No continuous integration or deployment pipeline is currently configured. No GitHub Actions workflows, Jenkinsfile, or equivalent pipeline configuration files exist in the repository. Adding automated CI/CD is identified as a near-term improvement priority.

11. Testing and Validation

11.1 Current Testing Status

The repository contains no automated test suites. No unit tests, integration tests, or end-to-end tests have been written. The project currently relies entirely on manual testing, using the platform's own built-in health endpoint, frontend health check indicator, and auto-generated API documentation.

11.2 Manual Testing Capabilities

The health endpoint at GET /health checks backend, HexStrike Server, and database connectivity in a single request. The home page displays a backend connection status indicator that reflects this health check. FastAPI auto-generates Swagger UI at /docs and ReDoc at /redoc, allowing developers to test API endpoints interactively. On the frontend, an ErrorBoundary

component catches React rendering errors and displays stack traces, and a SafeComponent wrapper provides component-level error handling. Backend routers return structured error responses with appropriate HTTP status codes.

11.3 Known Limitations

Several known issues are present in the codebase. There is no unit, integration, or end-to-end test coverage, and no load testing has been performed. The LoginPage contains multiple console.log statements that are more appropriate for debugging than for a released product. Some code duplication exists: routers/websocket_handler.py duplicates services/websocket_manager.py; services/result_parser.py exists but is not used by the tools router, which implements its own inline parser; and models/schemas.py is empty because schemas are defined inline in the routers. Celery and Redis dependencies are installed but not used. The ai_explanations_json column is defined in the database schema but is never written to.

12. Future Enhancements

12.1 Immediate Improvements

The most pressing improvement is to connect the LLMService to API endpoints, enabling the five AI capabilities to function in production. Completing the Docker containerisation files would significantly improve deployment reliability. Adding automated testing using pytest for the backend and Vitest or React Testing Library for the frontend would provide a safety net for future changes. Setting up a CI/CD pipeline with GitHub Actions for linting, testing, building, and deploying would further improve code quality and release confidence.

12.2 Scalability Improvements

Migrating from SQLite to PostgreSQL would enable reliable multi-user support, as SQLite's write-locking behaviour is unsuitable for concurrent production use. The already-installed Redis and Celery dependencies could be activated to provide session caching, rate limiting, and a proper background task queue. Implementing Alembic database migrations would allow schema changes to be applied safely as the project evolves.

12.3 Feature Enhancements

Several feature additions have been identified as valuable for future development. Auto-generated PDF reports of penetration testing sessions would provide a practical output users could retain and share. A multiplayer capture-the-flag mode would introduce competitive challenges between users. Real-time collaboration through shared terminal sessions would support group learning scenarios. A global XP leaderboard with weekly challenges would further motivate learners. A certificate system could award completion certificates at key course milestones. Replacing HTTP polling with full WebSocket-based streaming for tool output would improve the real-time experience. Finally, enforcing the module prerequisite definitions that are currently tracked but not enforced would add structure to the learning path.

13. Conclusion

HexStrike AI is a comprehensive cybersecurity education platform that brings together real penetration testing tool execution, AI-powered learning assistance, and gamified progress tracking into a single, cohesive web application. Its three-tier architecture, built on a React SPA, a FastAPI backend, and a service layer comprising SQLite, HexStrike Server, and OpenAI, provides a functional and extensible foundation for interactive security training.

The platform's key accomplishments include 42 registered security tools executable from the browser with real-time progress feedback; ten structured learning modules with embedded quizzes, flashcard decks, and lab exercises; a gamification system with XP rewards, five achievements, and activity streaks; a rule-based hint assistant and prepared AI service infrastructure; a clean REST API design with JWT authentication; and a dark-themed, responsive user interface suited to the cybersecurity domain.

The platform is functional for local educational use, but it remains in an early stage of development with respect to production readiness. Docker containerisation, automated testing, CI/CD pipelines, and a production-suitable database are all planned but not yet implemented. The codebase is well-structured and the architecture is sound, making these improvements achievable additions rather than fundamental redesigns. With the AI service connected and the testing and deployment gaps addressed, HexStrike AI would be well positioned as a genuinely useful tool for cybersecurity education.

14. Appendices

14.1 Appendix A: Complete File Structure

The repository follows a clear separation between frontend and backend concerns. The root-level directory contains the shell scripts for orchestration, the Docker placeholder files, and the Python environment setup script. The backend/ directory houses the FastAPI application, organised into routers/ (endpoint definitions), services/ (business logic), models/ (database models), and a configuration module. The frontend/ directory contains the Vite-based React application, with pages and components in the src/ directory. The sibling hexstrike-server/ repository contains the HexStrike Server that executes the penetration testing tools.

14.2 Appendix B: API Endpoint Summary

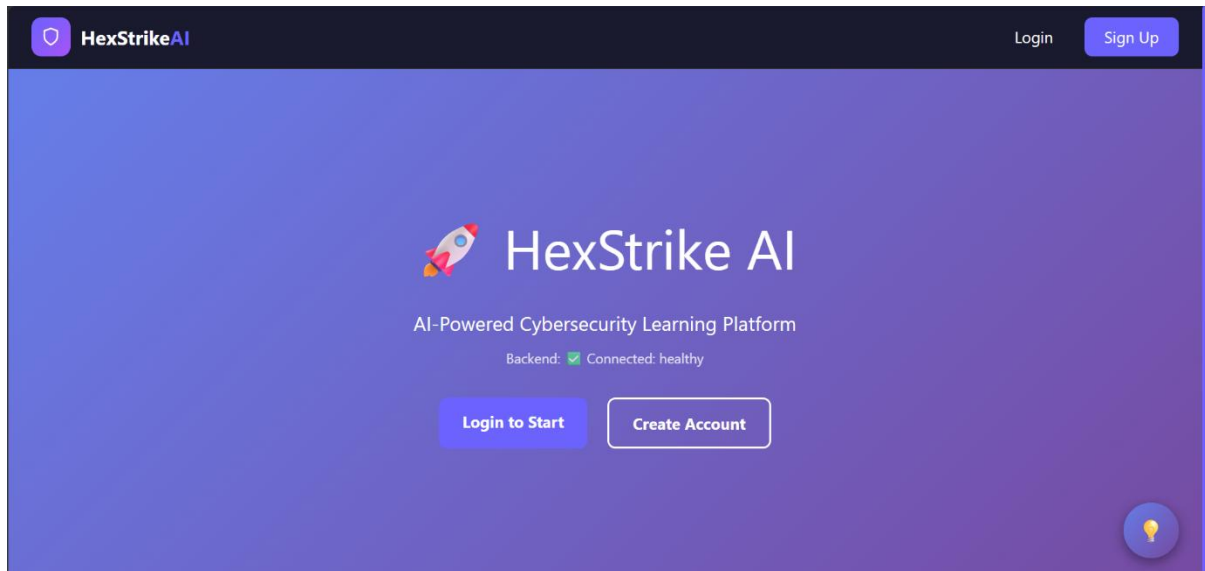
The full API is documented interactively at the /docs and /redoc endpoints when the backend server is running. The five router groups and their primary endpoints are: /api/auth/ for authentication (register, token, me); /api/tools/ for tool execution (execute, batch, status, history, delete); /api/learning/ for educational content (modules, content, quiz submission, flashcards, quiz history); /api/progress/ for user progress (retrieve and update XP, achievements, streaks); and root-level endpoints for health checking and WebSocket connections.

14.3 Appendix C: Glossary

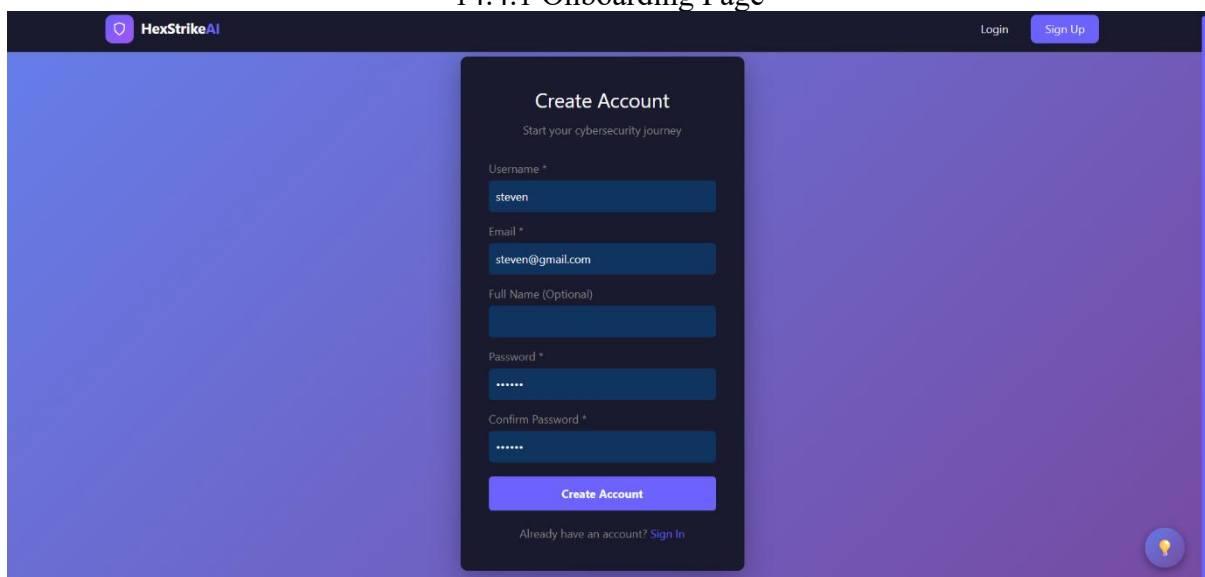
The following terms are used throughout this report. API (Application Programming Interface) refers to a defined set of endpoints that allow software components to communicate. ASGI (Asynchronous Server Gateway Interface) is the Python web server standard used by FastAPI and Uvicorn. CTF (Capture the Flag) is a competitive cybersecurity challenge format. JWT (JSON Web Token) is a compact, URL-safe token format used for authentication. ORM (Object-Relational Mapper) is a library that translates between database rows and programming

language objects. SPA (Single-Page Application) is a web application architecture in which the browser loads a single HTML page and updates content dynamically. XP (Experience Points) refers to the numerical reward system used in the gamification framework.

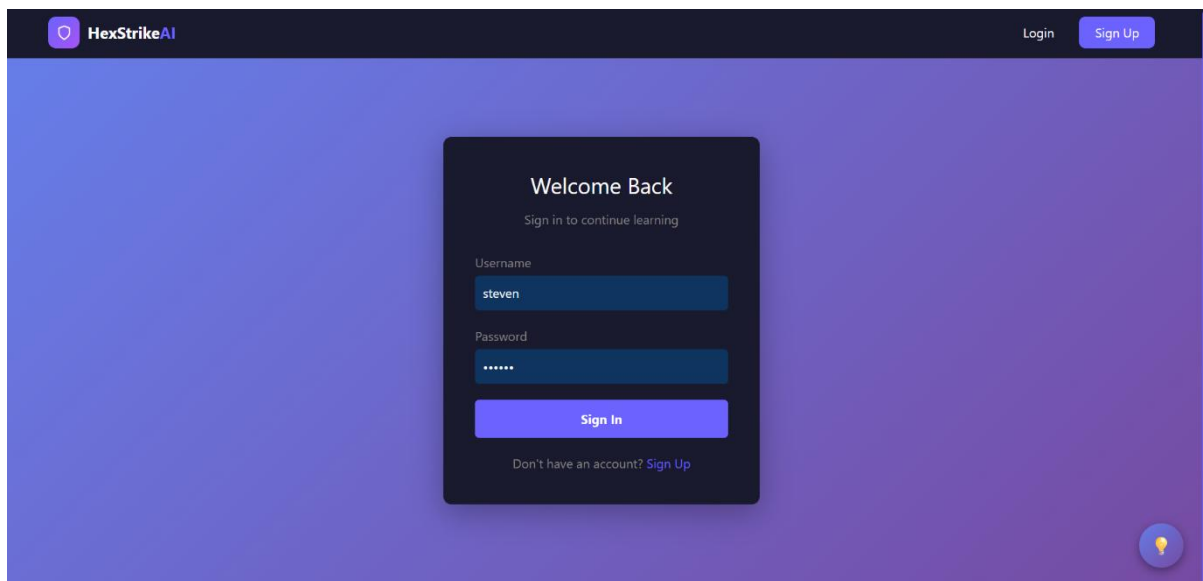
14.4 Appendix D: Screenshots



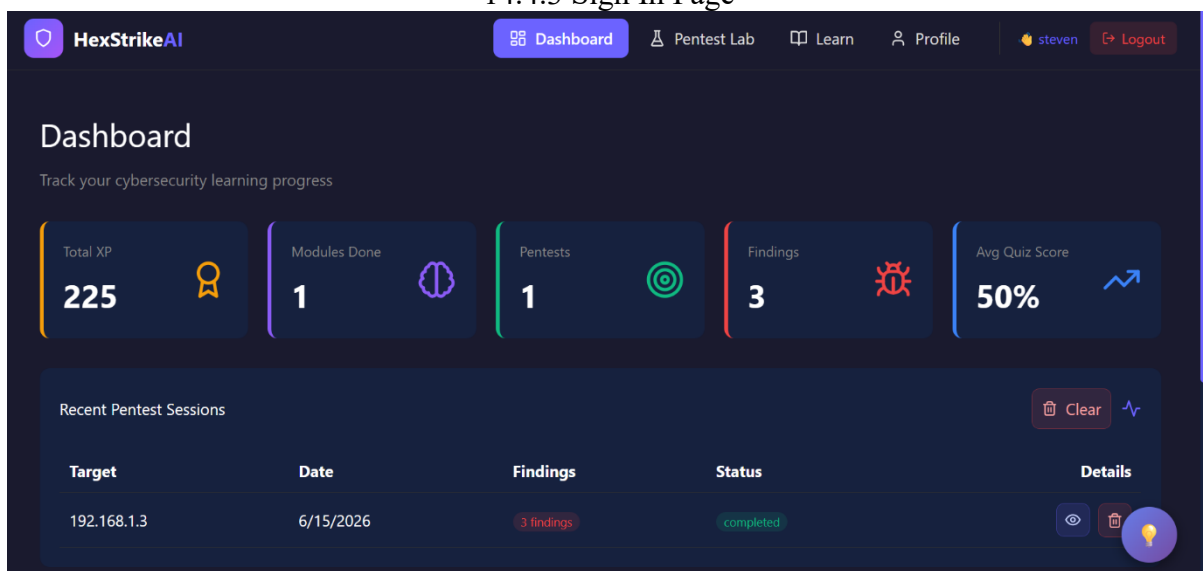
14.4.1 Onboarding Page



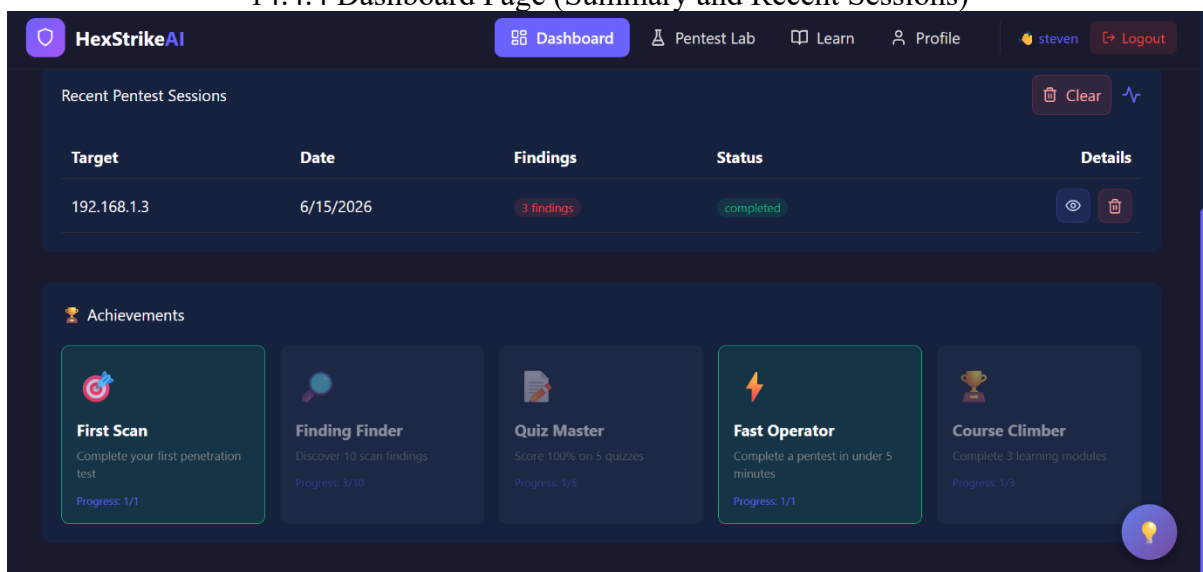
14.4.2 Create Account Page



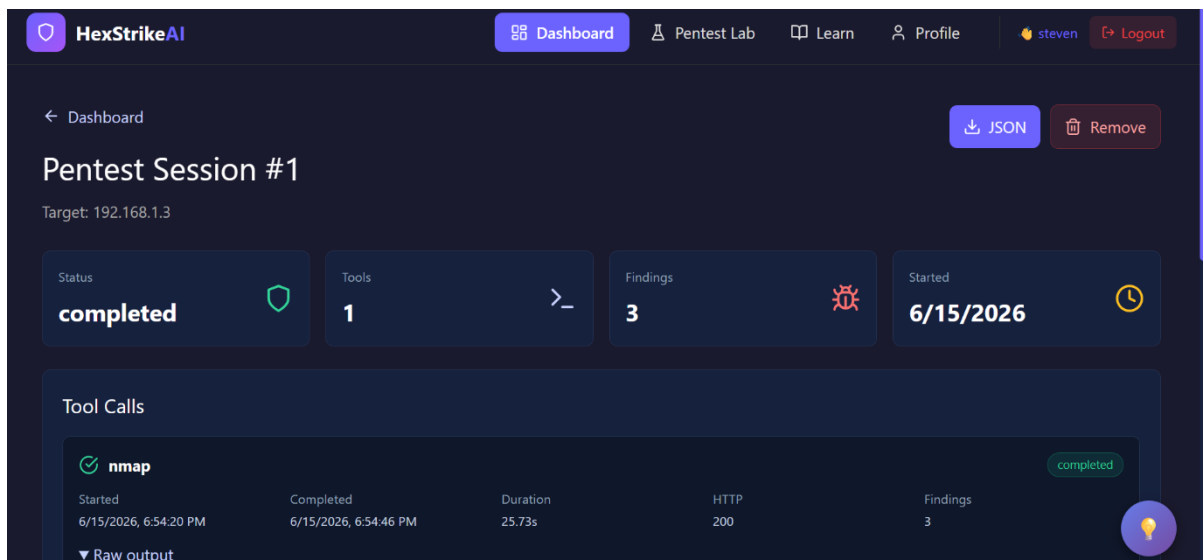
14.4.3 Sign In Page



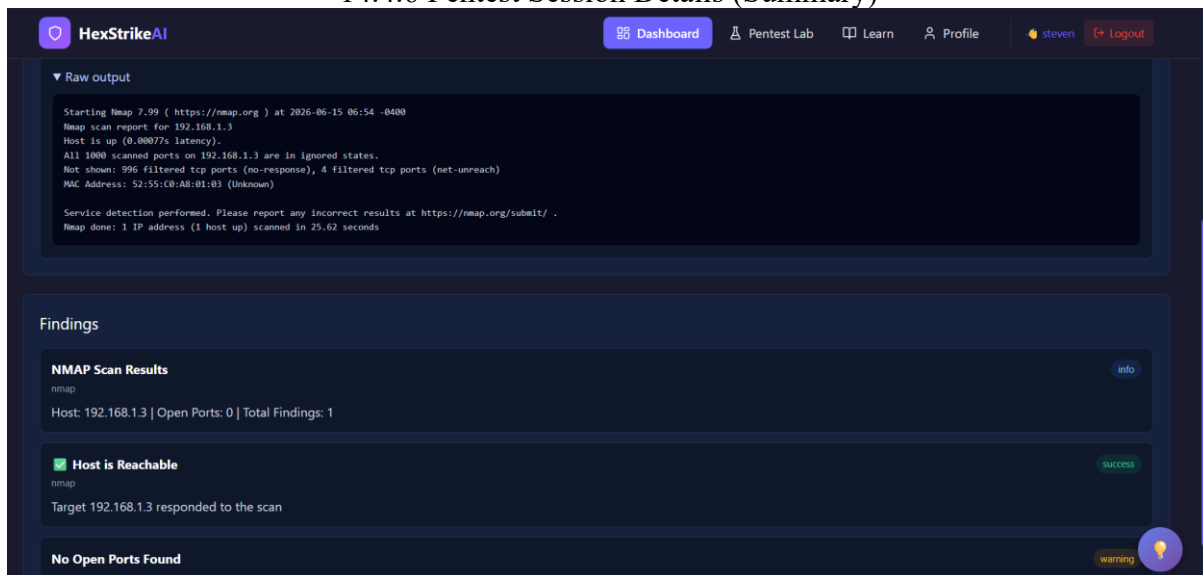
14.4.4 Dashboard Page (Summary and Recent Sessions)



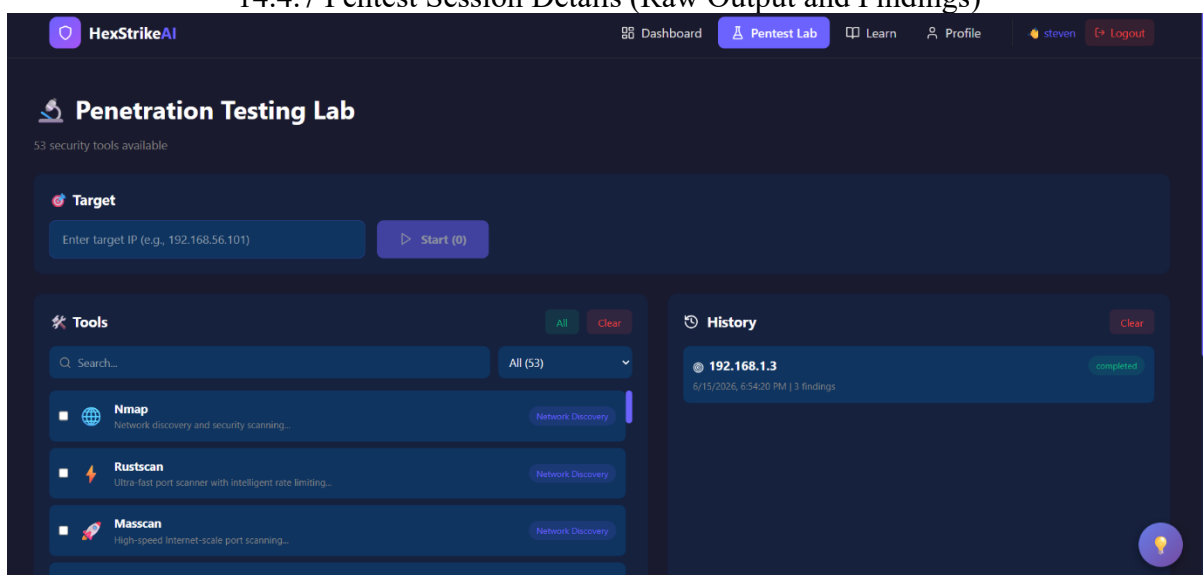
14.4.5 Dashboard Page (Achievements)



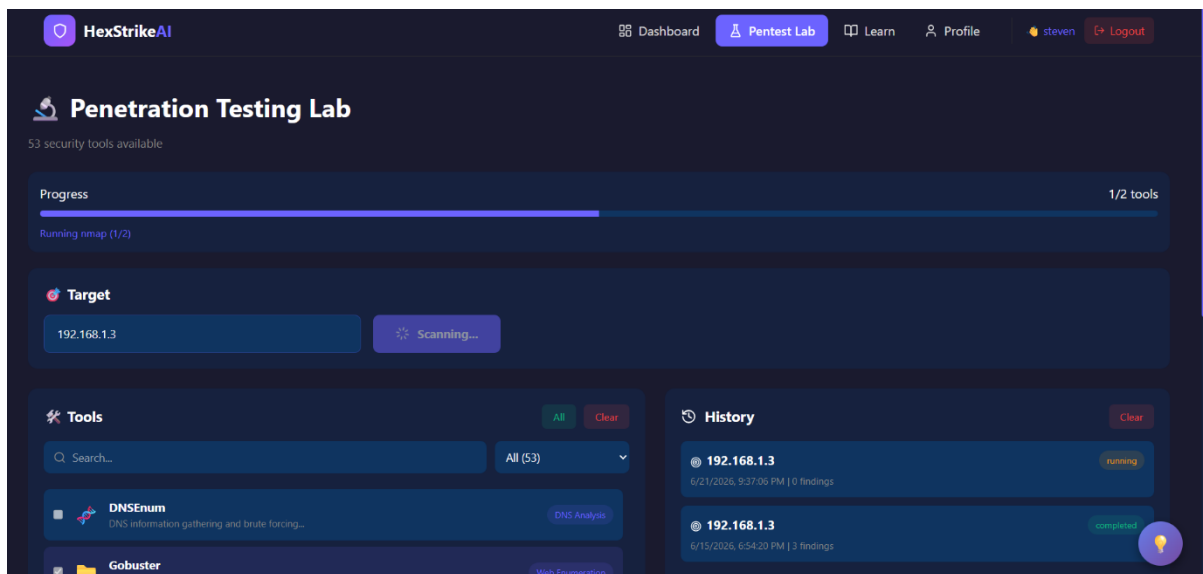
14.4.6 Pentest Session Details (Summary)



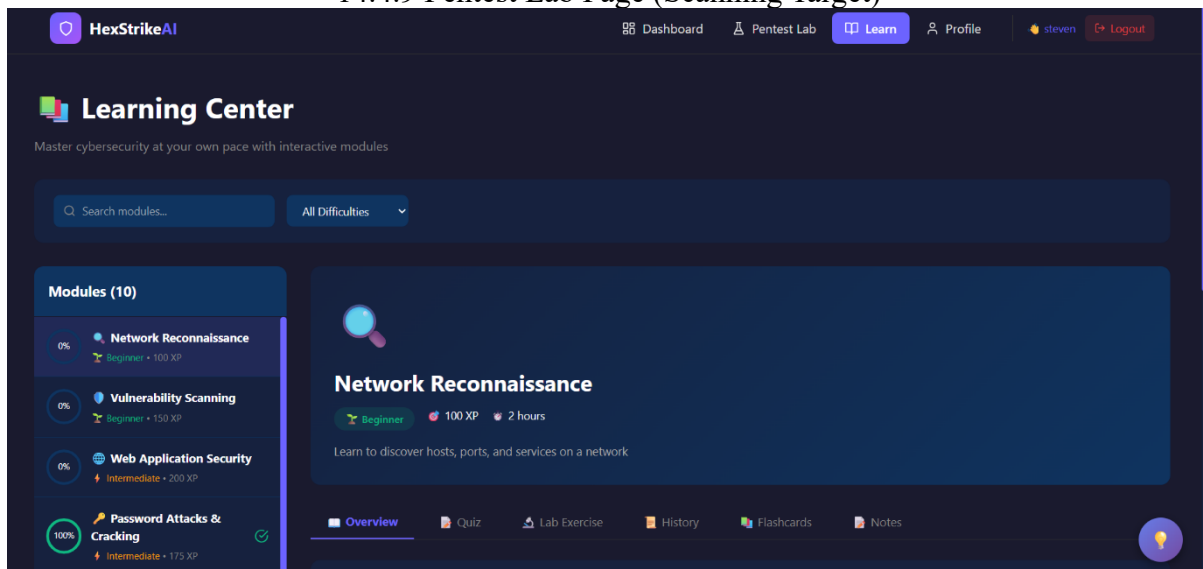
14.4.7 Pentest Session Details (Raw Output and Findings)



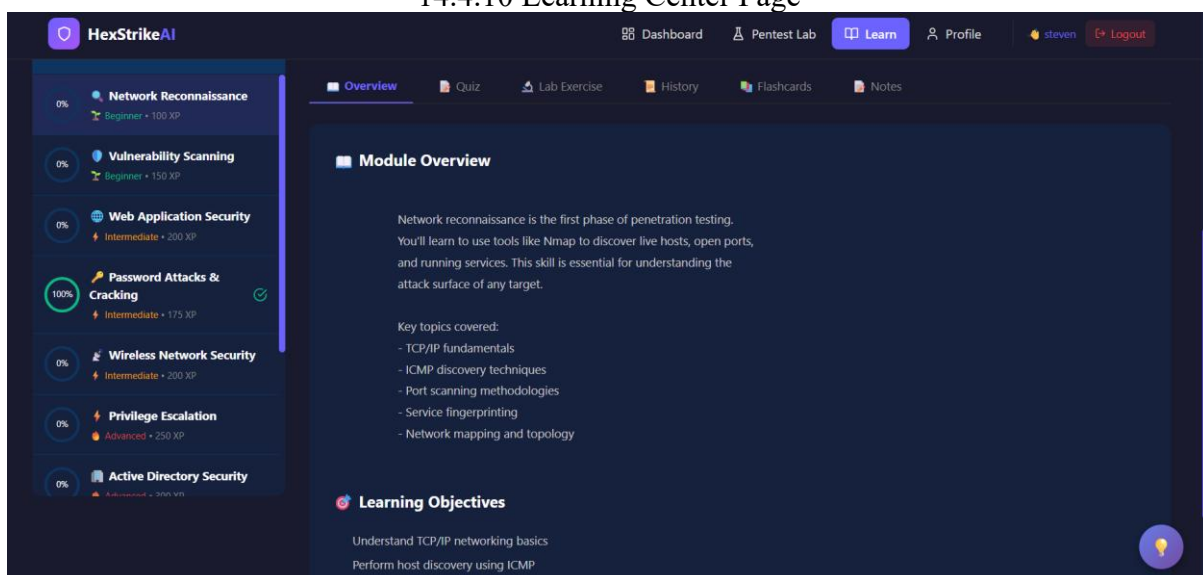
14.4.8 Pentest Lab Page



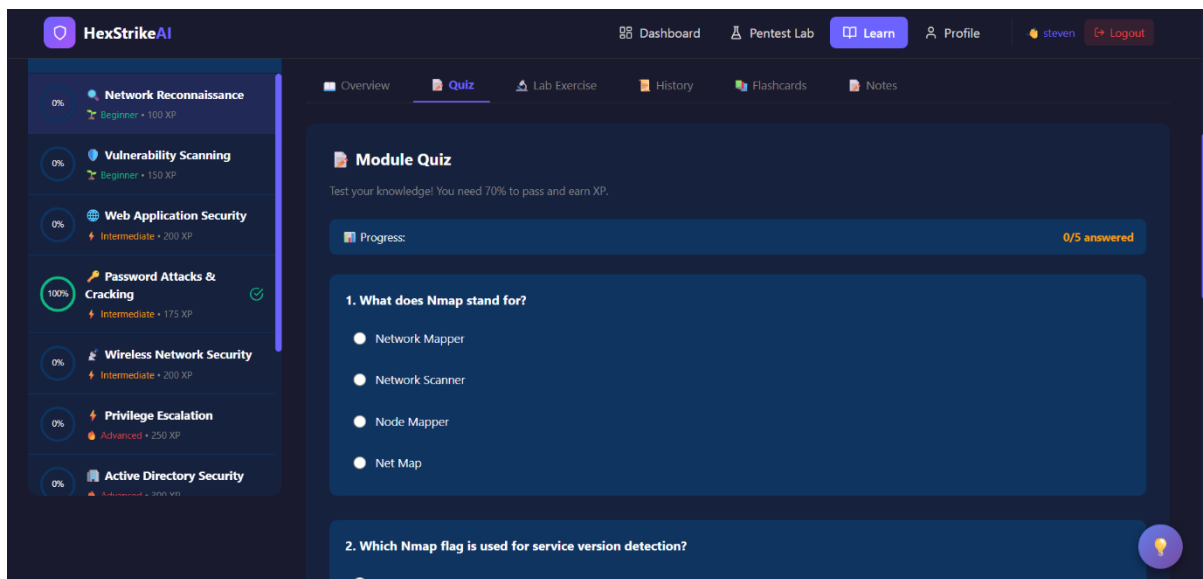
14.4.9 Pentest Lab Page (Scanning Target)



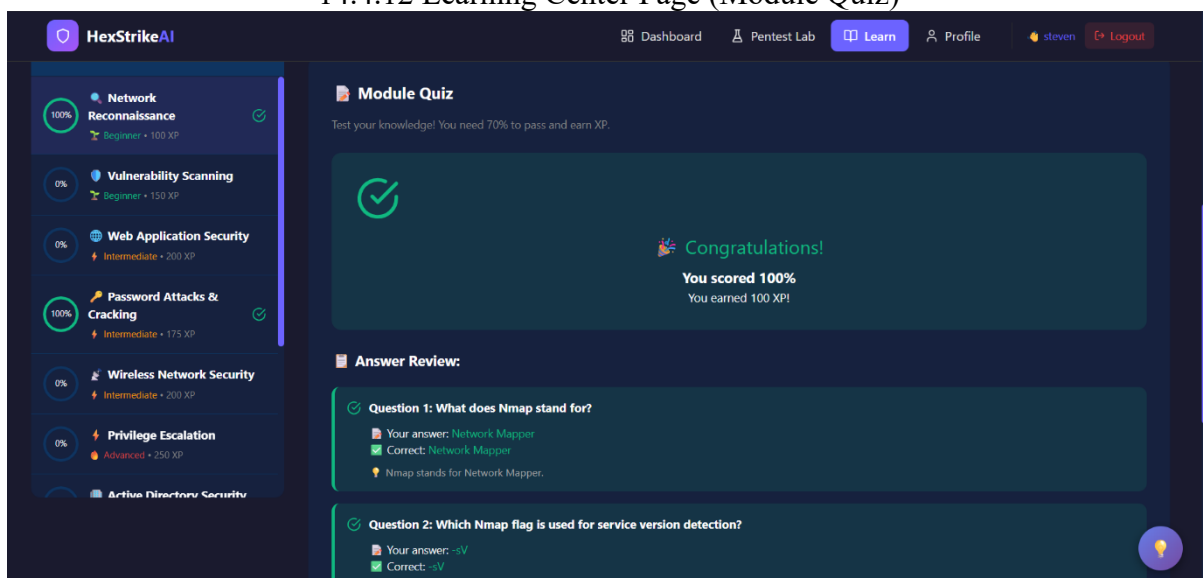
14.4.10 Learning Center Page



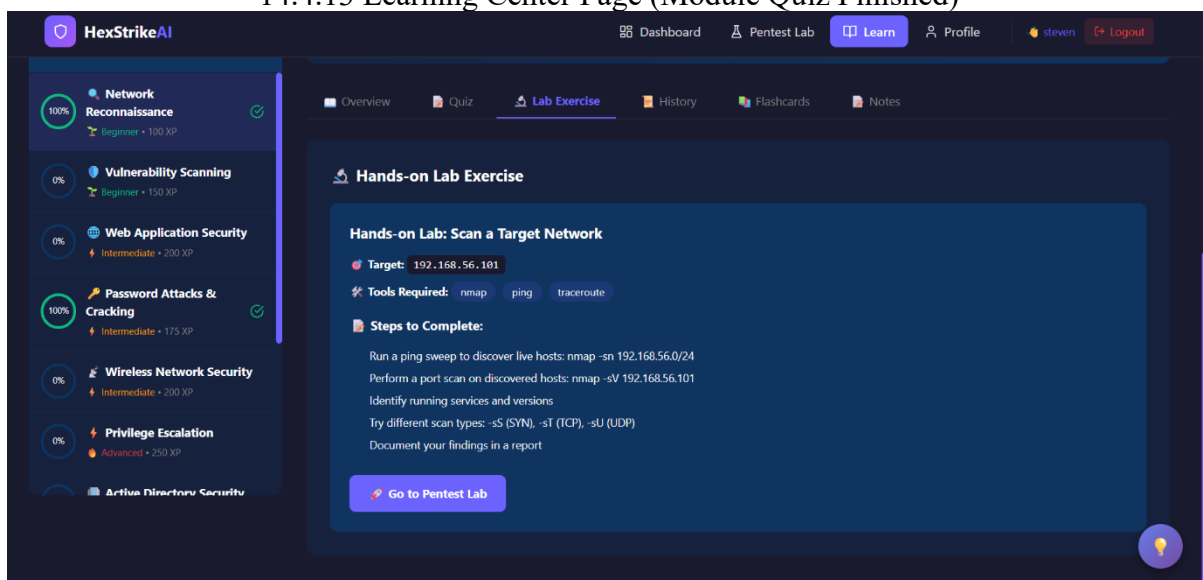
14.4.11 Learning Center page (Overview)



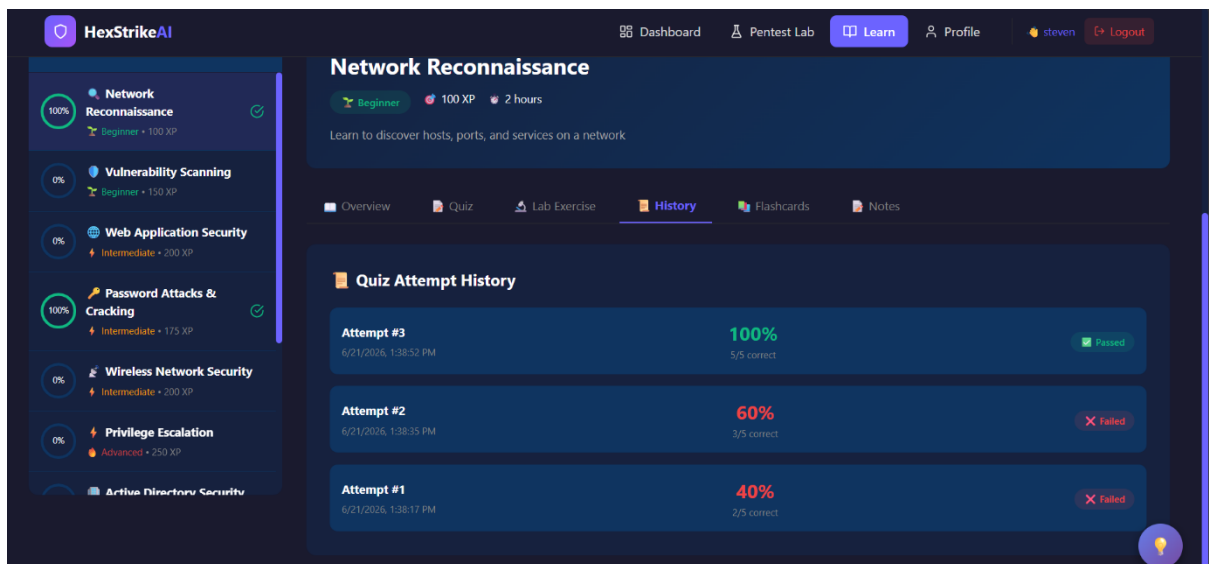
14.4.12 Learning Center Page (Module Quiz)



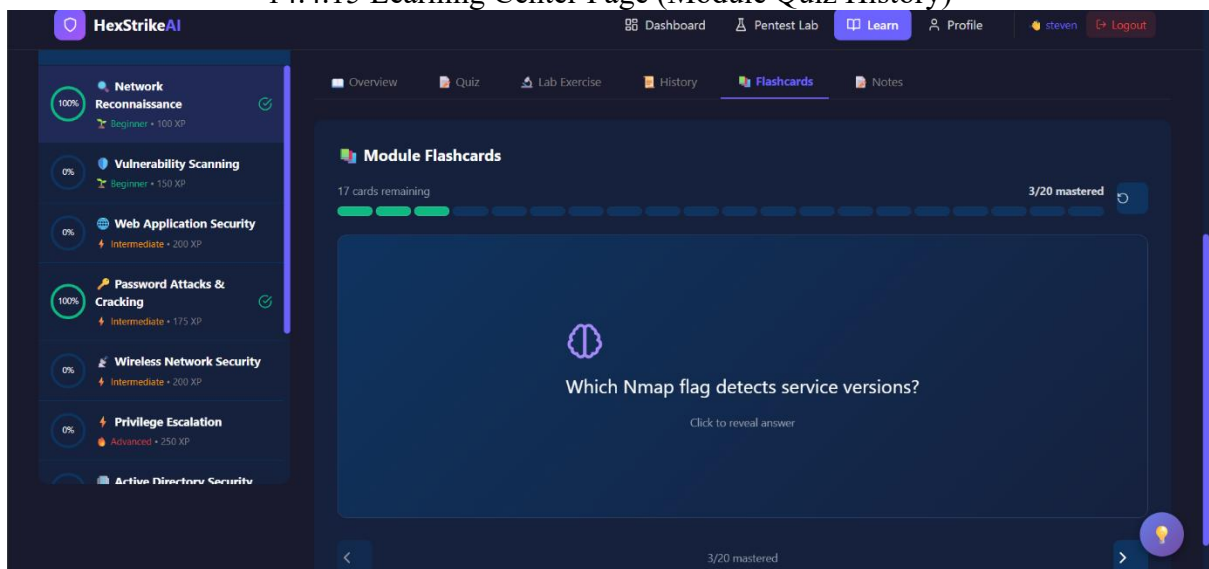
14.4.13 Learning Center Page (Module Quiz Finished)



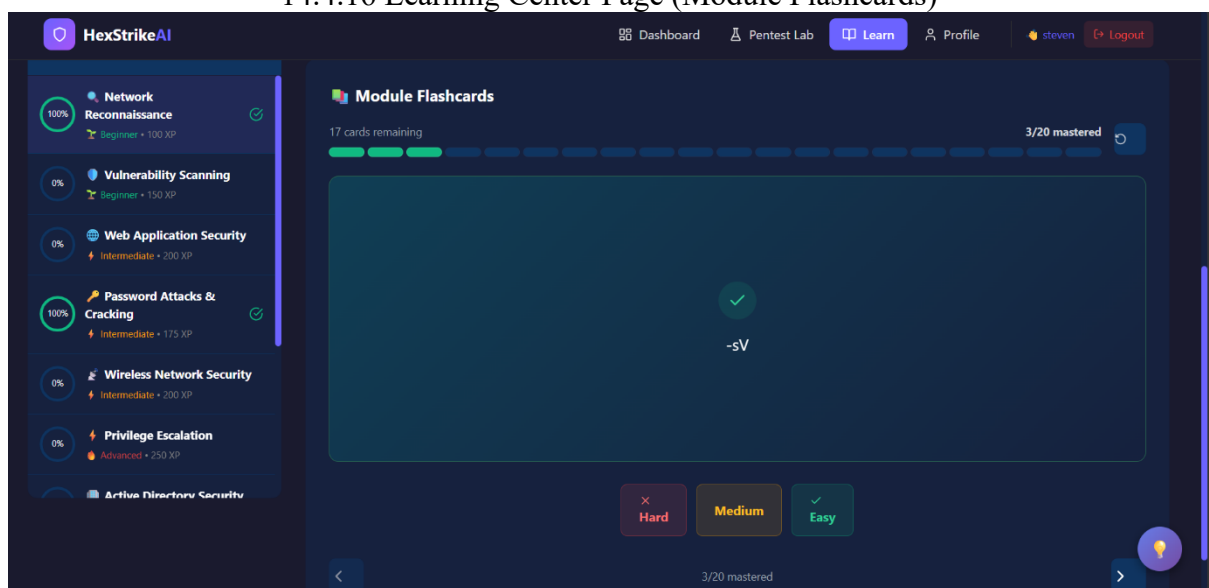
14.4.14 Learning Center Page (Lab Exercise)



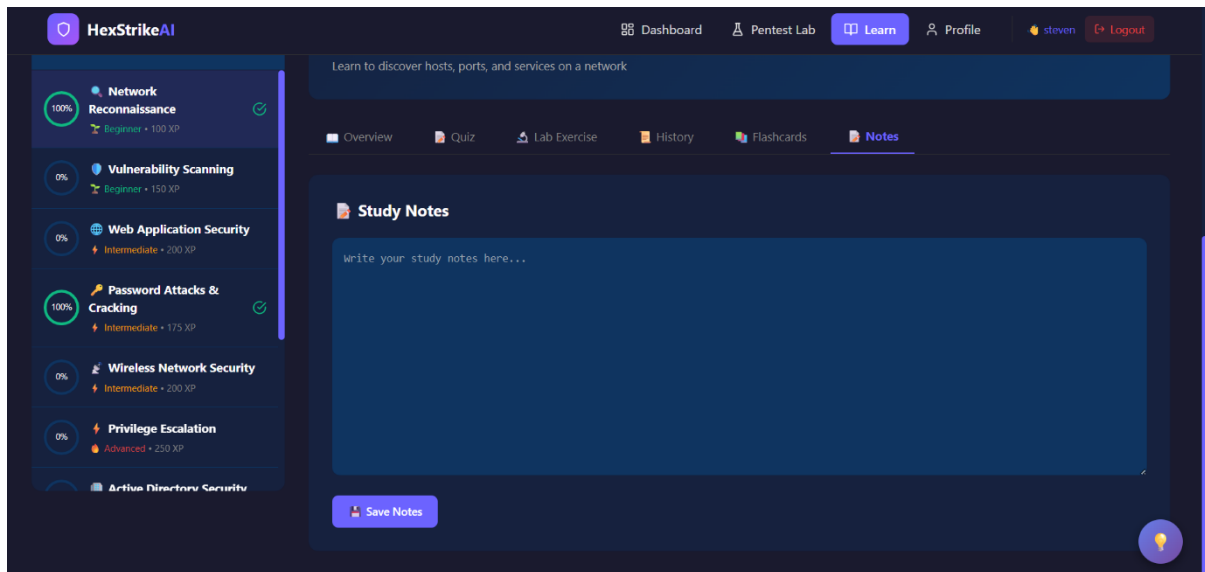
14.4.15 Learning Center Page (Module Quiz History)



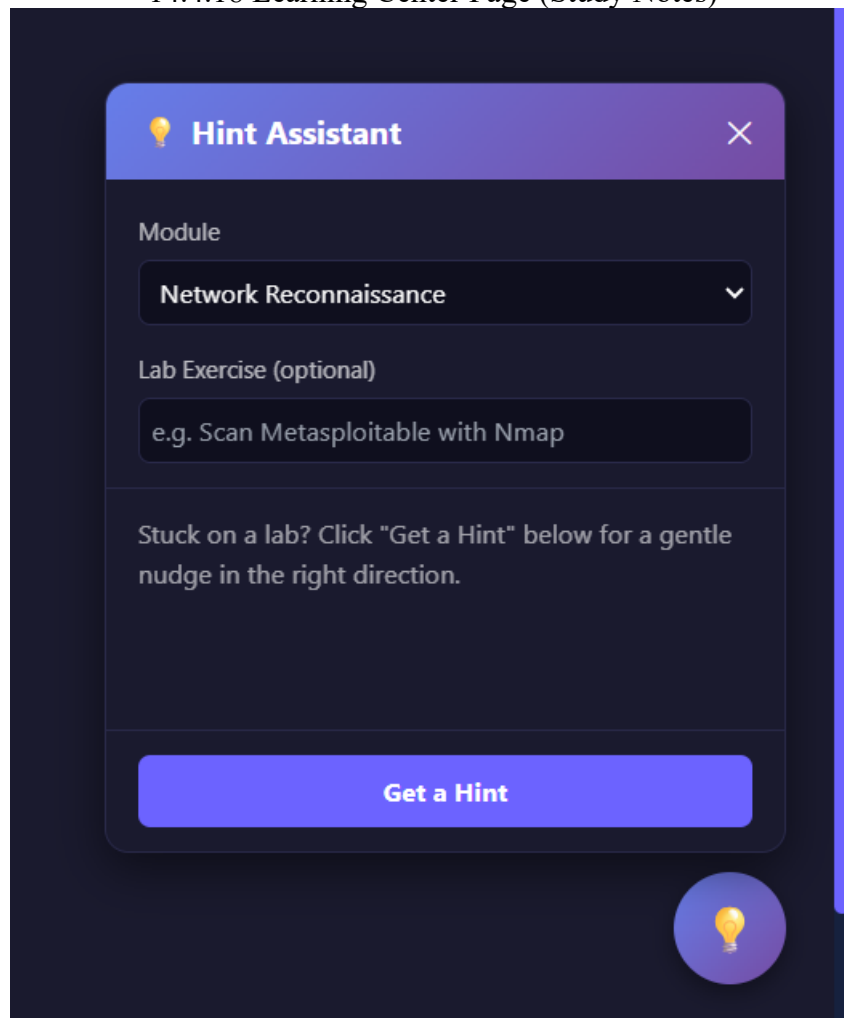
14.4.16 Learning Center Page (Module Flashcards)



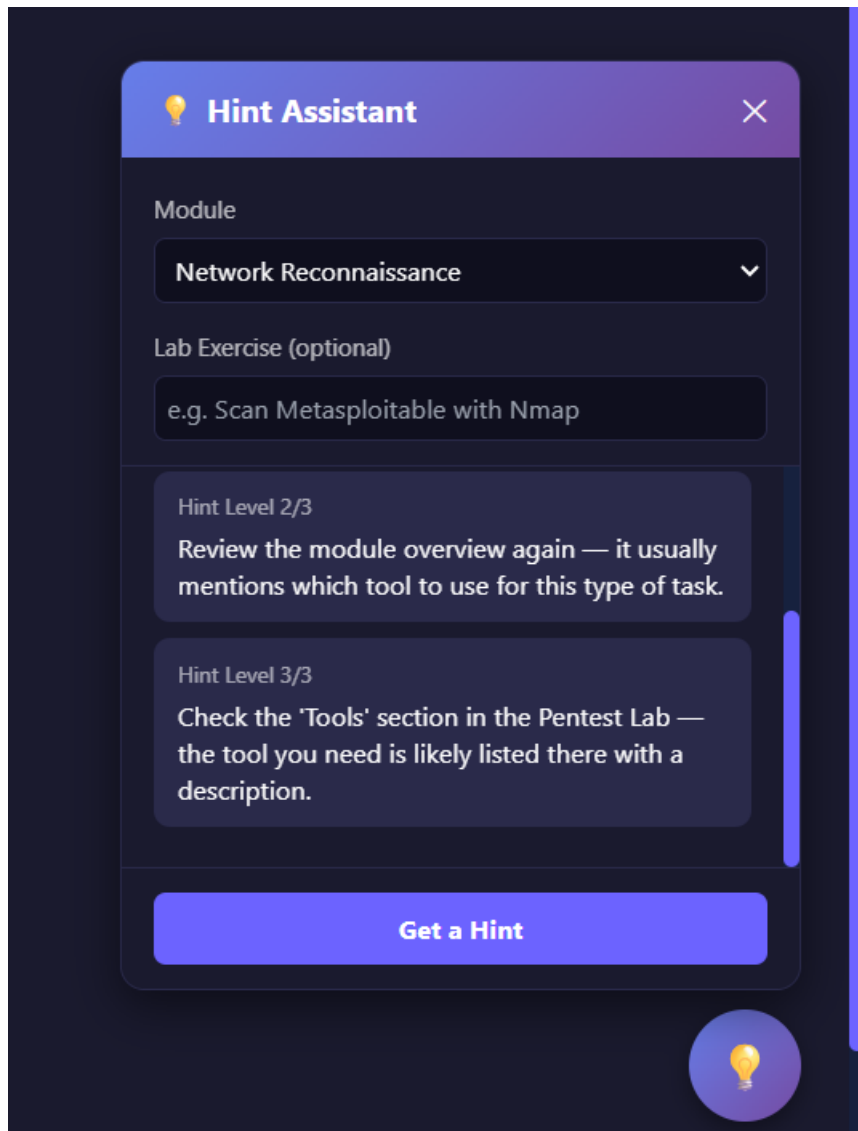
14.4.17 Learning Center Page (Module Flashcards Card Flipped)



14.4.18 Learning Center Page (Study Notes)



14.4.19 Hint Assistant Modal (Empty)



14.4.20 Hint Assistant Modal (Hint Shown)